



PROJECT REPORT

TITLE: Design and Implementation of a Database Management System for Alumni Mentoring and Engagement Management System



NAME: Sashrika Reddy Alimity

ROLL NO. : 25WU0102248

COURSE NAME: Database Management Systems

FACULTY NAME: Dr Kiran Mayee Adavala

ACADEMIC YEAR: 2025-2029

B. TECH AI ML

ABSTRACT

The Alumni Mentoring and Engagement Management System is a web-based database application developed to manage student profiles, alumni profiles, mentoring activities, career information, events, donations and feedback in a centralized relational database. The system is designed to organize the interactions between students, alumni and the institution while reducing the difficulties associated with maintaining these records manually.

In a traditional approach, student details, alumni information, employment records, mentoring requests, session information, event participation, donations and feedback may be maintained separately. This can result in duplication, inconsistency and difficulty in retrieving or updating information. The proposed system provides a structured solution using a relational database and a professional web interface.

The project uses HTML, CSS and JavaScript for the user interface and is designed around a MySQL relational database. The database contains thirteen interconnected tables: Program, Student, Alumni, Employment, Expertise_Area, Alumni_Expertise, Mentor_Request, Mentor_Assignment, Mentor_Session, Event, Event_Registration, Donation and Feedback. Primary keys and foreign keys are used to maintain relationships and preserve referential integrity.

The system demonstrates important DBMS concepts including relational schema design, normalization up to Third Normal Form (3NF), constraints, SQL queries, joins, aggregate functions, CRUD operations, searching, reporting and database connectivity. The UI also demonstrates insertion and deletion of records and provides modules for mentor search, requests, sessions, events, donations, feedback and reports.

INTRODUCTION

An alumni network contains valuable academic, professional and mentoring resources for an institution. Managing student profiles, alumni career information, expertise, mentoring relationships, events, donations and feedback manually becomes difficult as the number of users and interactions increases.

A Database Management System (DBMS) provides an organized solution by storing related information in separate tables and connecting those tables through primary and foreign keys. In the proposed Alumni Mentoring and Engagement Management System, Program is related to Student and Alumni. Alumni information is connected to Employment and Expertise_Area through Alumni_Expertise. Mentoring activities are represented through Mentor_Request, Mentor_Assignment and Mentor_Session, while Events, Event_Registration, Donation and Feedback support wider alumni engagement.

The system is intended to support the complete mentoring and engagement workflow, beginning with student and alumni registration, followed by mentor search, mentoring requests, mentor assignment and session scheduling. It also supports event participation, donation entry and feedback collection.

A web-based interface makes the system easier to use. Administrators can view records, insert new records, delete records, search for mentors and access reports without repeatedly entering SQL commands. This demonstrates how a relational database can be integrated with an application interface for a practical institutional use case.

PROBLEM STATEMENT

Managing alumni mentoring and engagement information manually becomes difficult when the number of students, alumni and activities increases. Student profiles, alumni details, employment history, expertise, mentor requests, mentoring sessions, events, donations and feedback may become distributed across separate files or systems.

One major challenge is finding a suitable mentor for a student. A student may require guidance in a particular expertise area, but without a structured database it is difficult to search alumni by expertise, employment or professional background. Mentoring requests and assignments can also become difficult to track.

Another challenge is maintaining consistency between related records. A mentor assignment should be connected to the correct request and alumni record, while a mentoring session should be connected to the correct assignment. Event registrations must reference valid events and students or alumni. Donations and feedback must also be associated with the appropriate alumni or session.

Therefore, a centralized relational database is required to organize the complete alumni mentoring and engagement process. The proposed system addresses these problems by storing information in structured tables, maintaining relationships through foreign keys and providing a web interface for common CRUD and search operations.

OBJECTIVES

The major objectives of the proposed Alumni Mentoring and Engagement Management System are:

1. To manage student information efficiently by storing and organizing student profiles in a structured database.
2. To maintain alumni profiles and graduation information in a centralized system.
3. To maintain alumni employment and career records so that professional background can be used for mentoring.
4. To maintain expertise areas and alumni expertise relationships for mentor search and matching.
5. To allow students to create mentoring requests based on their required expertise.
6. To manage mentor assignments while maintaining a valid relationship between requests and alumni mentors.
7. To schedule and track mentoring sessions with date, duration, mode and session status.
8. To manage institutional events and student/alumni registrations.
9. To record alumni donations and maintain organized contribution information.
10. To store feedback submitted for mentoring sessions and support evaluation of mentoring quality.
11. To provide CRUD operations through a web interface so records can be created, viewed, searched and deleted easily.
12. To maintain data integrity using primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints where applicable.
13. To provide reports and queries that demonstrate practical use of joins, subqueries, aggregation, views and database search.

SCOPE OF THE PROJECT

The scope of the proposed Alumni Mentoring and Engagement Management System covers the major activities required for managing students, alumni, mentoring and wider alumni engagement.

Student Management

The system stores Student_ID, name, email, Program_ID and year of study. Student records can be added, viewed, searched and deleted through the interface. Student information is connected to Program and Mentor_Request.

Alumni Management

The system stores Alumni_ID, name, email, graduation year and Program_ID. Alumni profiles are connected to employment history and expertise areas and can act as mentors.

Employment and Expertise Management

Employment stores career information such as company, job title and employment dates. Expertise_Area stores the expertise categories, while Alumni_Expertise connects alumni to their expertise and proficiency.

Mentoring Management

Mentor_Request records the student's mentoring requirement. Mentor_Assignment connects the request with an alumni mentor. Mentor_Session stores session date, duration, mode and status.

Event Management

The Event table stores event name, date, venue and capacity. Event_Registration records student or alumni participation and attendance status.

Donation Management

Donation stores alumni contribution details including donation amount and donation date. Positive donation values are required by the project business rules.

Feedback Management

Feedback records the rating and comments associated with a mentoring session. This allows the institution to evaluate mentoring interactions.

Database Operations and Web Interface

The project provides basic CRUD operations such as creating, viewing, searching and deleting records through the web interface. The interface includes a dashboard, navigation sidebar, forms, tables, filters, status badges and action buttons.

Database Relationships

Primary keys uniquely identify records, while foreign keys establish relationships between Program, Student, Alumni, Employment, Expertise_Area, Alumni_Expertise, Mentor_Request, Mentor_Assignment, Mentor_Session, Event, Event_Registration, Donation and Feedback.

Overall, the project provides a centralized solution for alumni mentoring and engagement and demonstrates relational tables, keys, normalization, SQL operations, CRUD functionality and application-level interaction.

SOFTWARE AND HARDWARE REQUIREMENTS

The Alumni Mentoring and Engagement Management System requires a basic computer system with suitable software for developing the web interface, managing the relational database and executing the application.

Software Requirements

- Operating System: Windows 10/11, macOS or equivalent.
- Visual Studio Code: Used for writing and managing project files.
- Python: Preferred backend/application language for database connectivity.
- MySQL: Relational database management system used for storing project data.
- HTML5: Used to structure the web interface.
- CSS3: Used to design and format the interface.
- JavaScript: Used for client-side interaction such as navigation, forms, search and CRUD demonstration.
- MySQL Connector/Python: Used when the Python application communicates with MySQL.
- Google Chrome or another modern browser: Used to run and demonstrate the web interface.

Hardware Requirements

- Processor: Intel Core i3 or equivalent and above.
- RAM: Minimum 4 GB; 8 GB recommended.
- Storage: At least 10 GB available.
- Display: Standard laptop/desktop display.
- Keyboard and mouse: Required for development and interaction.
- Internet connectivity: Useful for installing packages and accessing development resources.

SYSTEM ARCHITECTURE AND WORKFLOW

The application follows a layered structure in which the user interacts with the web interface, application logic validates and processes requests, and MySQL stores the relational data.

Layer	Main Responsibility
User Interface	Dashboard, sidebar, forms, tables, search, filters and action buttons.
Application Logic	Validation, CRUD functions, mentor search, request/session handling and error handling.
Database Layer	MySQL tables, primary keys, foreign keys, constraints, queries and views.

Main Workflow

- Student or administrator accesses the dashboard.
- Student/alumni records are inserted and maintained.
- Student searches for a mentor using expertise information.
- Student creates a Mentor_Request.
- Administrator/alumni coordinator assigns a suitable Alumni mentor.
- Mentor_Session records are created for the assignment.
- Students and alumni register for Event records.
- Alumni donations and mentoring feedback are recorded.
- Reports and SQL queries provide summarized information.

Business Rules

- Student and alumni email addresses must be unique.
- Donation amount must be positive.
- Mentoring sessions must use valid assignment records and valid session dates.
- Event registrations must reference valid events and the event capacity must not be exceeded.
- Only one active mentor assignment should exist for a mentoring request.

ER DIAGRAM

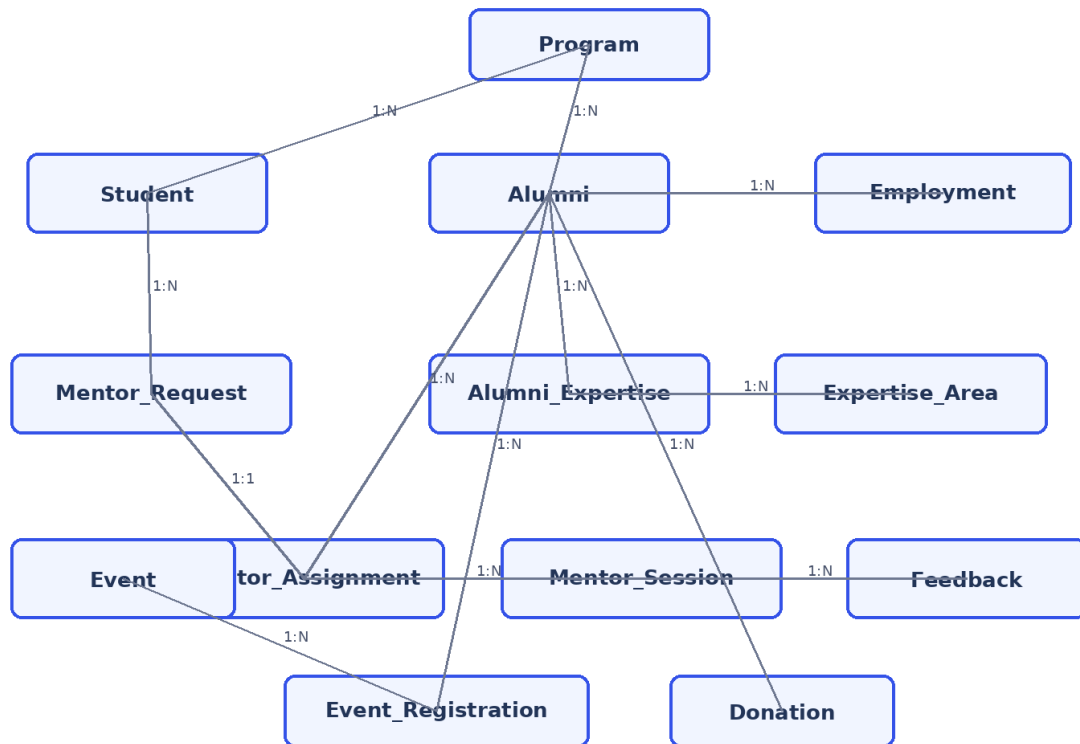


Figure: Conceptual ER diagram for the Alumni Mentoring and Engagement Management System.

The ER design contains thirteen entities/tables. Program has one-to-many relationships with Student and Alumni. Alumni connects to Employment and to Expertise_Area through Alumni_Expertise. Student creates Mentor_Request, which can lead to Mentor_Assignment and Mentor_Session. Mentoring sessions can receive Feedback. Alumni and students can participate in Event through Event_Registration, and Alumni can make Donations.

RELATIONAL SCHEMA AND NORMALIZATION

Relational Schema

The relational schema defines the structure of the database by specifying tables, attributes, primary keys and foreign keys. The database consists of thirteen interconnected relations.

Table	Primary Key	Important Foreign Keys
PROGRAM	program_id	—
STUDENT	student_id	program_id → Program
ALUMNI	alumni_id	program_id → Program
EMPLOYMENT	employment_id	alumni_id → Alumni
EXPERTISE_AREA	expertise_id	—
ALUMNI_EXPERTISE	(alumni_id, expertise_id)	both → Alumni / Expertise_Area
MENTOR_REQUEST	request_id	student_id, expertise_id
MENTOR_ASSIGNMENT	assignment_id	request_id, alumni_id
MENTOR_SESSION	session_id	assignment_id
EVENT	event_id	—
EVENT_REGISTRATION	registration_id	event_id, student_id, alumni_id
DONATION	donation_id	alumni_id
FEEDBACK	feedback_id	session_id

RELATIONAL SCHEMA — KEY RELATIONSHIPS

PROGRAM

PROGRAM(program_id PK, program_name, department)

STUDENT

STUDENT(student_id PK, name, email UNIQUE, program_id FK, year_of_study)

ALUMNI

ALUMNI(alumni_id PK, name, email UNIQUE, graduation_year, program_id FK)

EMPLOYMENT

EMPLOYMENT(employment_id PK, alumni_id FK, company_name, job_title, start_date, end_date)

EXPERTISE_AREA

EXPERTISE_AREA(expertise_id PK, expertise_name UNIQUE)

ALUMNI_EXPERTISE

ALUMNI_EXPERTISE(alumni_id PK/FK, expertise_id PK/FK, proficiency_level)

MENTOR_REQUEST

MENTOR_REQUEST(request_id PK, student_id FK, expertise_id FK, request_date, description, status)

MENTOR_ASSIGNMENT

MENTOR_ASSIGNMENT(assignment_id PK, request_id FK, alumni_id FK, assigned_date, status)

MENTOR_SESSION

MENTOR_SESSION(session_id PK, assignment_id FK, session_date, duration_minutes, mode, session_status)

EVENT

EVENT(event_id PK, event_name, event_date, venue, capacity)

EVENT_REGISTRATION

EVENT_REGISTRATION(registration_id PK, event_id FK, student_id FK, alumni_id FK, registration_date, attendance_status)

DONATION

DONATION(donation_id PK, alumni_id FK, amount, donation_date)

FEEDBACK

FEEDBACK(feedback_id PK, session_id FK, rating, comments)

NORMALIZATION

Normalization is the process of organizing data to reduce redundancy, prevent update anomalies and improve consistency. The project database is designed to satisfy Third Normal Form (3NF).

First Normal Form — 1NF

A table is in 1NF when every attribute contains an atomic value and there are no repeating groups. In this project, each field stores a single value and each table has a unique primary key. For example, Student stores one email and one program_id per student.

Second Normal Form — 2NF

A table is in 2NF when it is in 1NF and every non-key attribute depends on the complete primary key. Tables with simple primary keys have attributes dependent on their row identifier. Alumni_Expertise uses a composite primary key of alumni_id and expertise_id, while proficiency_level depends on the complete combination.

Third Normal Form — 3NF

A table is in 3NF when it is in 2NF and non-key attributes do not depend on other non-key attributes. Program details are stored in Program rather than duplicated in Student and Alumni. Employment details are stored separately from Alumni. Expertise information is stored in Expertise_Area and linked through Alumni_Expertise.

Normalization therefore reduces duplication, simplifies updates and maintains consistency across the mentoring and engagement database.

DATA DICTIONARY

The data dictionary describes the fields, data types and constraints used in the Alumni Mentoring and Engagement Management System.

Table	Field	Data Type	Constraint / Description
Program	program_id	INT	PK
	program_name	VARCHAR(100)	NOT NULL
	department	VARCHAR(100)	NOT NULL
Student	student_id	INT	PK
	name	VARCHAR(100)	NOT NULL
	email	VARCHAR(150)	UNIQUE, NOT NULL
	program_id	INT	FK, NOT NULL
	year_of_study	INT	NOT NULL
Alumni	alumni_id	INT	PK
	name	VARCHAR(100)	NOT NULL
	email	VARCHAR(150)	UNIQUE, NOT NULL
	graduation_year	INT	NOT NULL
	program_id	INT	FK, NOT NULL
Employment	employment_id	INT	PK
	alumni_id	INT	FK, NOT NULL
	company_name	VARCHAR(100)	NOT NULL
	job_title	VARCHAR(100)	NOT NULL
	start_date	DATE	NOT NULL
	end_date	DATE	Nullable

DATA DICTIONARY — CONTINUED

Table	Field	Data Type	Constraint / Description
Expertise_Area	expertise_id	INT	PK
	expertise_name	VARCHAR(100)	UNIQUE, NOT NULL
Alumni_Expertise	alumni_id	INT	PK/FK
	expertise_id	INT	PK/FK
	proficiency_level	VARCHAR(50)	Nullable
Mentor_Request	request_id	INT	PK
	student_id	INT	FK, NOT NULL
	expertise_id	INT	FK, NOT NULL
	request_date	DATE	NOT NULL
	description	VARCHAR(255)	Nullable
	status	VARCHAR(20)	DEFAULT PENDING
Mentor_Assignment	assignment_id	INT	PK
	request_id	INT	FK, NOT NULL
	alumni_id	INT	FK, NOT NULL
	assigned_date	DATE	NOT NULL
	status	VARCHAR(20)	DEFAULT ACTIVE
Mentor_Session	session_id	INT	PK
	assignment_id	INT	FK, NOT NULL
	session_date	DATE	NOT NULL
	duration_minutes	INT	NOT NULL
	mode	VARCHAR(20)	NOT NULL
	session_status	VARCHAR(20)	DEFAULT SCHEDULED
Event	event_id	INT	PK
	event_name	VARCHAR(100)	NOT NULL
	event_date	DATE	NOT NULL
	venue	VARCHAR(100)	NOT NULL
	capacity	INT	NOT NULL
Event_Registration	registration_id	INT	PK
	event_id	INT	FK, NOT NULL
	student_id	INT	FK, Nullable
	alumni_id	INT	FK, Nullable
	registration_date	DATE	NOT NULL
	attendance_status	VARCHAR(20)	DEFAULT REGISTERED
Donation	donation_id	INT	PK
	alumni_id	INT	FK, NOT NULL
	amount	DECIMAL(10,2)	NOT NULL; positive
	donation_date	DATE	NOT NULL
Feedback	feedback_id	INT	PK
	session_id	INT	FK, NOT NULL
	rating	INT	NOT NULL
	comments	VARCHAR(255)	Nullable

SQL COMMANDS

DDL COMMAND

DDL (Data Definition Language) commands are used to create and define the structure of the database and its tables. In this project, DDL is used for creating the relational structure, keys and constraints required by the Alumni Mentoring and Engagement Management System.

```
CREATE TABLE Student (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  program_id INT NOT NULL,  
  year_of_study INT NOT NULL,  
  FOREIGN KEY (program_id)  
  REFERENCES Program(program_id)  
);
```

OUTPUT: The Student table is created with a primary key, unique email constraint and foreign key relationship to Program.

```
CREATE TABLE Mentor_Request (  
  request_id INT PRIMARY KEY,  
  student_id INT NOT NULL,  
  expertise_id INT NOT NULL,  
  request_date DATE NOT NULL,  
  description VARCHAR(255),  
  status VARCHAR(20) DEFAULT 'PENDING',  
  FOREIGN KEY (student_id) REFERENCES Student(student_id),  
  FOREIGN KEY (expertise_id) REFERENCES Expertise_Area(expertise_id)  
);
```

DML COMMAND AND QUERIES WITH OUTPUTS

DML COMMAND

DML (Data Manipulation Language) commands are used to manage records stored in the database. The application uses INSERT, SELECT, UPDATE and DELETE operations.

```
INSERT INTO Student
(student_id, name, email, program_id, year_of_study)
VALUES (151, 'Sashrika Demo',
'sashrika.demo@university.edu', 1, 2);
```

```
UPDATE Mentor_Request
SET status = 'ASSIGNED'
WHERE request_id = 1;
```

Query 1: SELECT — Display student records

```
SELECT student_id, name, email, program_id, year_of_study
FROM Student
ORDER BY student_id
LIMIT 10;
```

student_id	name	email	program_id	year_of_study
1	Aarav	student1@gmail.com	1	1
2	Ananya	student2@gmail.com	2	2
3	Rahul	student3@gmail.com	3	3
4	Priya	student4@gmail.com	4	4

QUERIES WITH OUTPUTS

Query 2: WHERE — Find alumni with AI expertise

```
SELECT a.name, a.email, e.expertise_name
FROM Alumni a
JOIN Alumni_Expertise ae ON a.alumni_id = ae.alumni_id
JOIN Expertise_Area e ON ae.expertise_id = e.expertise_id
WHERE e.expertise_name = 'Artificial Intelligence';
```

Query 3: JOIN — Display mentor request details

```
SELECT s.name AS student, e.expertise_name,
mr.request_date, mr.status
FROM Mentor_Request mr
JOIN Student s ON mr.student_id = s.student_id
JOIN Expertise_Area e ON mr.expertise_id = e.expertise_id;
```

Query 4: GROUP BY / Aggregate — Count alumni by company

```
SELECT company_name, COUNT(*) AS alumni_count
FROM Employment
GROUP BY company_name
ORDER BY alumni_count DESC;
```

These queries demonstrate selection, joins and aggregation over the project's normalized relations.

QUERY GIVEN DURING PRESENTATION-II

QUESTION: Retrieve the details of students who requested Artificial Intelligence expertise and have made donations of 25,000 or above through related alumni engagement records.

CODE:

```
SELECT DISTINCT s.name AS Student_Name,  
a.name AS Alumni_Name,  
e.expertise_name,  
d.amount  
FROM Mentor_Request mr  
JOIN Student s ON mr.student_id = s.student_id  
JOIN Expertise_Area e ON mr.expertise_id = e.expertise_id  
JOIN Mentor_Assignment ma ON mr.request_id = ma.request_id  
JOIN Alumni a ON ma.alumni_id = a.alumni_id  
JOIN Donation d ON a.alumni_id = d.alumni_id  
WHERE e.expertise_name = 'Artificial Intelligence'  
AND d.amount >= 25000;
```

OUTPUT: The query returns only matching student/alumni relationships where the mentoring request concerns Artificial Intelligence and the related alumni donation is at least ₹25,000.

UI DESIGN

Dashboard and Student Management

The web interface follows a professional dashboard layout with a fixed blue sidebar, white top navigation area, dashboard statistics, quick actions, tables, filters and forms. The design is adapted specifically for the Alumni Mentoring and Engagement Management System.

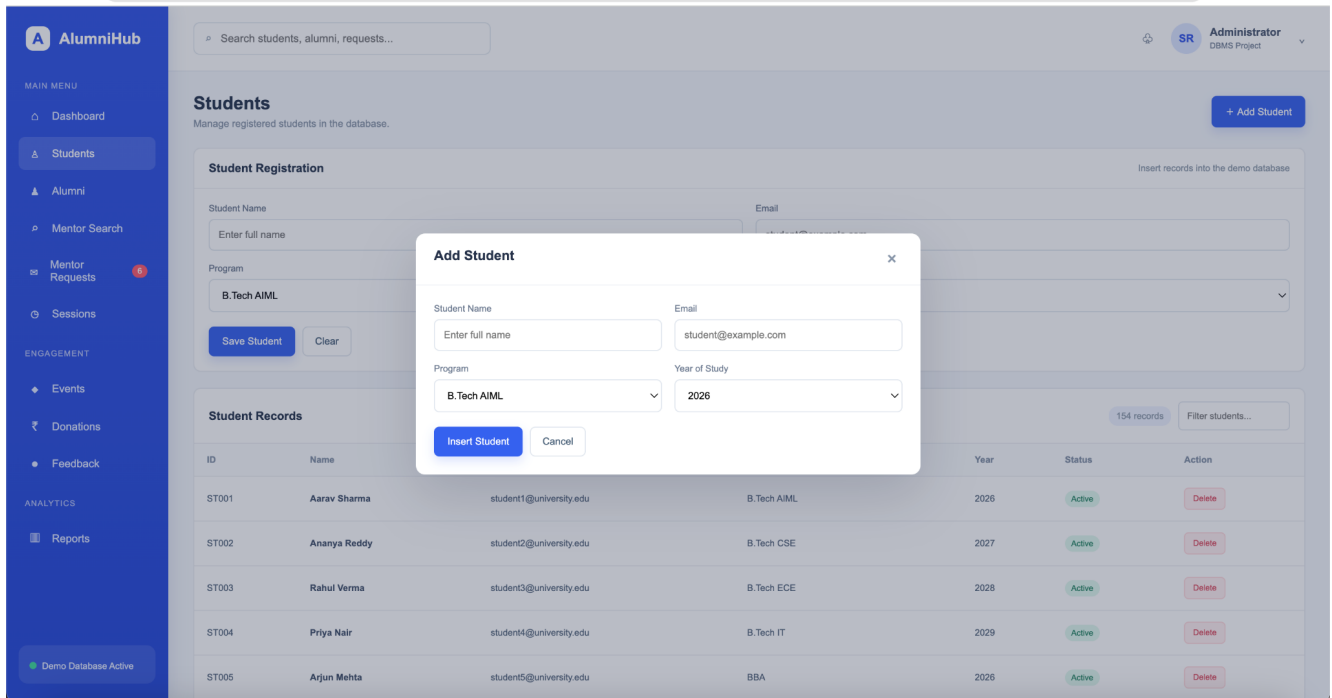


Figure: AlumniHub student management interface with sidebar navigation and Add Student form.

The screenshot shows the Student module, including the Dashboard/Students navigation, Student Registration form and Student Records table. This directly demonstrates the CRUD-oriented UI used for the project.

01. VIEW OPERATION / 02. INSERTION OPERATION

ID	Name	Email	Program	Year	Status	Action
ST004	Priya Nair	student4@university.edu	B.Tech IT	2029	Active	Delete
ST005	Arjun Mehta	student5@university.edu	BBA	2026	Active	Delete
ST006	Kavya Rao	student6@university.edu	MBA	2027	Active	Delete

Figure: Student records displayed in the AlumniHub interface.

The screenshot shows the AlumniHub interface. On the left is a sidebar with navigation links: Dashboard, Students, Alumni, Mentor Search, Mentor Requests, Sessions, Events, Donations, Feedback, and Reports. The main content area is titled 'Students' and includes a search bar, a '+ Add Student' button, and a table of student records. The 'Add Student' modal is open, showing fields for Student Name, Email, Program, and Year of Study. The modal also has 'Insert Student' and 'Cancel' buttons. The background table shows student records with columns: ID, Name, Email, Program, Year, Status, and Action.

Figure: Add Student modal used for insertion.

The Student table displays multiple records with Student ID, name, email, program, year, status and Delete action. The Add Student modal collects the fields required for insertion. The UI validates required name/email fields and checks duplicate email values before insertion.

03. DELETION OPERATION

Each Student and Alumni record contains a Delete action. When the user selects Delete, the interface asks for confirmation and then removes the selected record from the active browser-side dataset. The dashboard count and table are refreshed immediately.

ID	Name	Email	Program	Year	Status	Action
ST004	Priya Nair	student4@university.edu	B.Tech IT	2029	Active	Delete
ST005	Arjun Mehta	student5@university.edu	BBA	2026	Active	Delete
ST006	Kavya Rao	student6@university.edu	MBA	2027	Active	Delete

Figure: Student records with Delete controls.

IMPLEMENTATION DETAILS

The demonstration UI is implemented as a standalone HTML file using HTML5, CSS3 and JavaScript. It contains the dashboard, sidebar navigation, forms, tables, mentor search and CRUD demonstration. For the final database-connected implementation, Python can be used as the application layer with MySQL Connector/Python to communicate with the MySQL database.

IMPLEMENTATION DETAILS

Technology Stack

The Alumni Mentoring and Engagement Management System is designed as a web-based database application using frontend, application and database technologies.

Technology	Purpose
HTML5	Designing the structure of web pages and forms
CSS3	Styling and formatting the professional dashboard interface
JavaScript	Client-side navigation, search, validation and CRUD demonstration
Python	Preferred backend/application development language
MySQL	Storing and managing the thirteen relational tables
MySQL Connector/Python	Connecting the Python application to MySQL
Visual Studio Code	Development environment

Database Connectivity

For the database-connected version, the Python application uses MySQL Connector/Python to establish a connection. SQL statements are executed through a cursor and changes are committed after successful insert, update or delete operations.

DATABASE CONNECTIVITY AND KEY CODE SNIPPETS

A basic MySQL connection for the final application follows the structure below:

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="alumni_mentoring_db"
)

cursor = db.cursor()
```

The connection object communicates with MySQL while the cursor executes SQL commands. After an insertion or update, commit() is used to save changes permanently.

Insert Student

```
query = '''
INSERT INTO Student
(student_id, name, email, program_id, year_of_study)
VALUES (%s, %s, %s, %s, %s)
'''

values = (student_id, name, email, program_id, year_of_study)
cursor.execute(query, values)
db.commit()
```

Delete Student

```
cursor.execute(
    "DELETE FROM Student WHERE student_id = %s",
    (student_id,)
)
db.commit()
```

Parameterized SQL values help separate user input from SQL syntax and support safer database operations.

TESTING RESULT

Testing was performed to verify that the Alumni Mentoring and Engagement Management System supports the expected database and UI operations. Each major feature was tested using suitable input and the actual result was compared with the expected result.

Test Case	Description	Expected Result	Actual Result	Status
TC01	Add a new student	Student details should be stored	Student record inserted successfully	Pass
TC02	View student details	Existing records should be displayed	Student details displayed correctly	Pass
TC03	Add a new alumni	Alumni profile should be added	Alumni record inserted successfully	Pass
TC04	Search mentor	Matching expertise profiles should appear	Matching alumni displayed	Pass
TC05	Create mentor request	Request should be linked to student/expertise	Request stored with PENDING status	Pass
TC06	Assign mentor	Request should connect to alumni	Assignment recorded	Pass
TC07	Schedule session	Valid session should be stored	Session displayed as scheduled	Pass
TC08	Delete student	Selected record should be removed	Student deleted successfully	Pass

TESTING RESULT — CONTINUED

Test Case	Description	Expected Result	Actual Result	Status
TC09	Register for event	Valid event registration should be stored	Registration stored successfully	Pass
TC10	Donation validation	Negative/zero donation should be rejected	Invalid amount rejected	Pass
TC11	Unique email validation	Duplicate student/alumni email should be rejected	Duplicate email rejected	Pass
TC12	Event capacity	Registration should not exceed capacity	Capacity rule validated	Pass
TC13	Feedback submission	Feedback should reference valid session	Feedback stored successfully	Pass
TC14	Reports	Queries should return expected summaries	Report data displayed	Pass

Validation and Error Handling

- Required fields are checked before insertion.
- Duplicate email addresses are rejected.
- Foreign-key relationships prevent invalid references in the MySQL database.
- Donation amounts are validated to be positive.
- Session and event rules are checked before accepting records.
- Delete actions require user confirmation in the demonstration UI.

CONCLUSION

The Alumni Mentoring and Engagement Management System was developed to provide a structured solution for managing students, alumni, mentoring activities and wider alumni engagement. The system organizes student profiles, alumni information, employment records, expertise, mentoring requests, assignments, sessions, events, registrations, donations and feedback in a centralized relational database.

The project demonstrates practical DBMS concepts including relational tables, primary keys, foreign keys, UNIQUE and NOT NULL constraints, normalization, SQL queries, joins, aggregate functions, CRUD operations and database connectivity. The thirteen-table structure provides clear separation of responsibilities while maintaining relationships through foreign keys.

The AlumniHub web interface provides a convenient way to view records, add students and alumni, delete records, search mentors and navigate through the different management modules. The dashboard gives a high-level overview of students, alumni, mentors and events, while dedicated modules support mentoring requests, sessions, events, donations, feedback and reports.

Overall, the project provides practical experience in designing and implementing a normalized relational database and connecting it to a user-facing application. It demonstrates how DBMS concepts can be applied to a real institutional problem involving alumni mentoring and engagement.

FUTURE ENHANCEMENTS

Although the current system provides the major functions required for managing alumni mentoring and engagement, several features can be added in the future.

- Secure student, alumni and administrator login with role-based access control.
- Real-time mentor recommendation using expertise, employment history and student requirements.
- Email or SMS notifications for mentor assignments, upcoming sessions and events.
- Calendar integration for mentoring sessions and institutional events.
- Advanced analytics dashboards for mentor match rates, session completion and alumni engagement.
- Online donation payment integration and automated donation receipts.
- Document or certificate upload for alumni professional profiles.
- Advanced search and filtering by company, role, expertise, graduation year and program.
- Automated event capacity enforcement and wait-list management.
- Deployment of the MySQL-backed application on a secure server so multiple users can access the same live database.
- Audit logging for important insert, update and delete operations.

REFERENCES

1. MySQL Documentation – MySQL Reference Manual for SQL commands, database creation, table management, constraints and queries.
2. Python Documentation – Python Software Foundation documentation for Python programming and database connectivity.
3. Flask Documentation – Reference for web application development and routing when the Python backend is deployed.
4. W3Schools – SQL, HTML, CSS, JavaScript and Python tutorials used for understanding programming and database concepts.
5. GeeksforGeeks – Reference material for DBMS concepts, SQL queries, normalization, joins and database operations.
6. MySQL Connector/Python Documentation – Reference for connecting Python applications with MySQL.
7. Project Review 1 and Review 2 materials – Alumni Mentoring and Engagement Management System database design, ER model, schema and requirements.
8. AlumniHub DBMS UI – Project demonstration interface created for student/alumni CRUD, mentor search and engagement management.

PROJECT REPOSITORY AND FINAL SUBMISSION

Project Title: Design and Implementation of a Database Management System for Alumni Mentoring and Engagement Management System

Student: Sashrika Reddy Alimity

Roll No: 25WU0102248

Course: Database Management Systems

Database: MySQL

Application: Web UI with HTML/CSS/JavaScript; Python/MySQL planned for database connectivity

Final Project Components

- MySQL database containing 13 normalized relational tables.
- ER diagram and relational schema.
- SQL DDL, DML, joins, subqueries, aggregation and reports.
- AlumniHub dashboard and CRUD-oriented UI.
- Student and Alumni insertion/deletion demonstration.
- Mentor search, mentor requests and session management modules.
- Event, donation and feedback modules.
- Testing and validation of important business rules.
- Project report and presentation materials.

GitHub Repository Link

THANK YOU

GitHub Repository Link: To be added